TAD – Listas Lineares

Prof. Fernando Garcia

Referências

Material adaptado do livro:

ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.

Os código presentes nos slides estão disponíveis no site do autor (link no último slide)

Listas Lineares

- Uma das formas mais simples de interligar os elementos de um conjunto.
- Estrutura em que as operações inserir, retirar e localizar são definidas.
- São estruturas flexíveis:
  - podem crescer ou diminuir de tamanho durante a execução de um programa.
- Itens de uma lista podem ser:
  - Acessados;
  - Inseridos;
  - retirados.

Listas Lineares

- Duas listas podem ser concatenadas para formar uma lista única, ou uma pode ser partida em duas ou mais listas.
- As listas são adequadas quando não é possível prever a demanda por memória, permitem:
  - Manipulação de quantidades imprevisíveis de dados.
  - Manipulação de dados de formato também imprevisível.

Listas Lineares

- São úteis em aplicações tais como manipulação simbólica, gerência de memória, simulação e compiladores.

- Uma lista linear é uma sequência de zero ou mais itens , na qual é de uma determinado tipo e representa o tamanho da lista linear.

Listas Lineares

- Sua principal propriedade estrutural envolve as posições relativas dos itens em uma dimensão.
  - Assumindo , o primeiro item da lista e é o último item da lista.
  - precede para
  - sucede para
  - o elemento é dito estar na -ésima posição da lista.

Operação sobre Listas Lineares

- Para criar uma TAD Lista, precisa-se definir um conjunto de operação.
- O conjunto de operações a ser definido depende de cada aplicação.
- No entanto, há um conjunto de operações necessário a uma maioria de aplicações, entre elas:
  - Criar uma lista linear vazia
  - Inserir um novo item imediatamente após o -ésimo item.
  - Retirar o -ésimo item.
  - Localizar o -ésimo item para examinar e/ou alterar o conteúdo de seus componentes.
  - Fazer uma cópia da lista linear.
  - Ordenar os itens da lista em ordem ascendente ou descendente, de acordo com alguns de seus componentes.
  - Pesquisar a ocorrência de um item com um valor particular em algum componente.

Representação de Listas Lineares

- Existem várias estruturas de dados que podem ser usadas para representar listas lineares:
- As duas representações mais utilizadas são:
  - por meio de arranjos; e
  - estruturas auto referenciadas (ou apontadores para linguagens como C e C++).

Representação de Listas Lineares

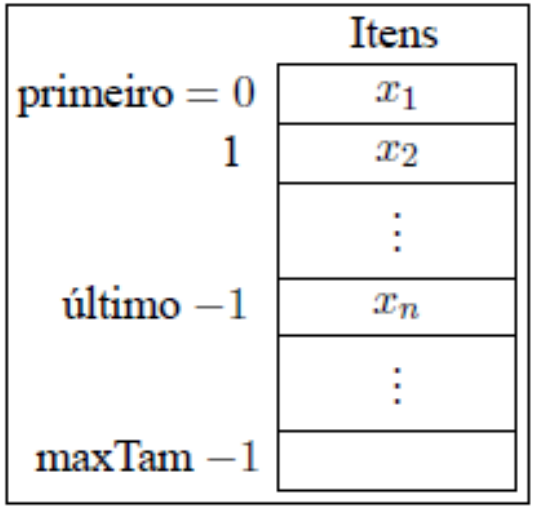
- Exemplos de Operações:
  - Lista(maxTam). Cria uma lista vazia.
  - insere(x). Insere após o último item da lista.
  - retira(x). Retorna o item x que está na posição p da lista, retirando-o da lista e deslocando os itens a partir da posição para as posições anteriores.
  - vazia(). Esta função retorna *true* se lista vazia; senão retorna *false*.
  - imprime(). Imprime os itens da lista na ordem de ocorrência.

Implementação de Listas por meio de Arranjos

- Os itens são armazenados em um arranjo de tamanho suficiente para armazenar a lista.
- Os itens da lista são armazenados em posições contíguas de memória.
- A lista pode ser percorrida em qualquer direção.
- Inserção de um novo item:
  - após o último item com custo constante.
  - no meio da lista requer um deslocamento de todos os itens localizados após o ponto de inserção.

Implementação de Listas por meio de Arranjos

- Retirar um item do início da lista requer um deslocamento de itens para preencher o espaço deixado vazio.



Referência: ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5						

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5	X_6					

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

Insere

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

prim ultim

A inserção padrão dos elementos na lista é na posição último. Porém, é possível também fazer inserção em qualquer posição.

Propriedade das Listas Lineares

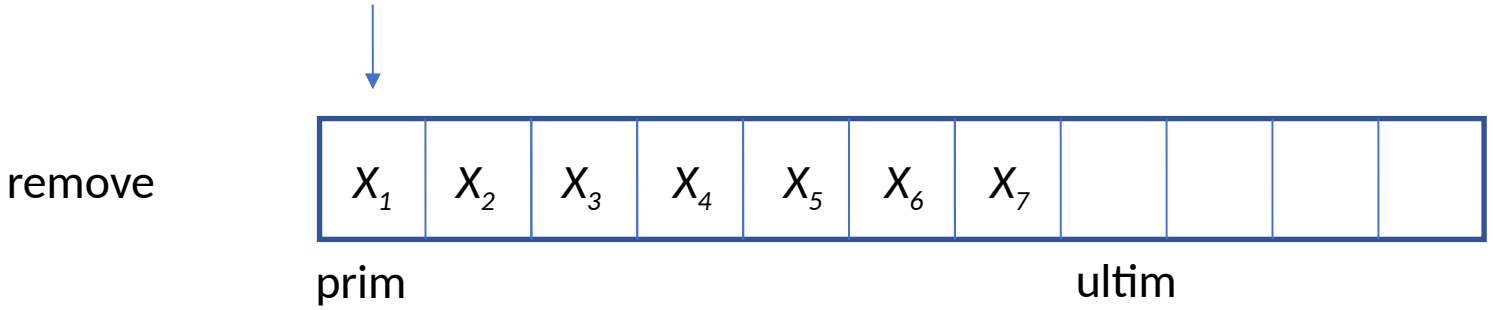
remove

X_1	X_2	X_3	X_4	X_5	X_6	X_7				

prim ultim

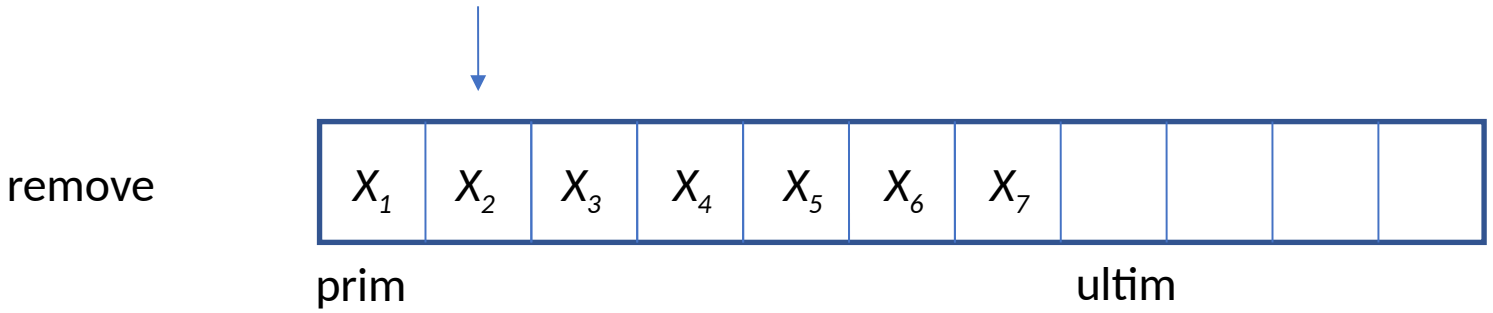
Remover: X_3

Propriedade das Listas Lineares



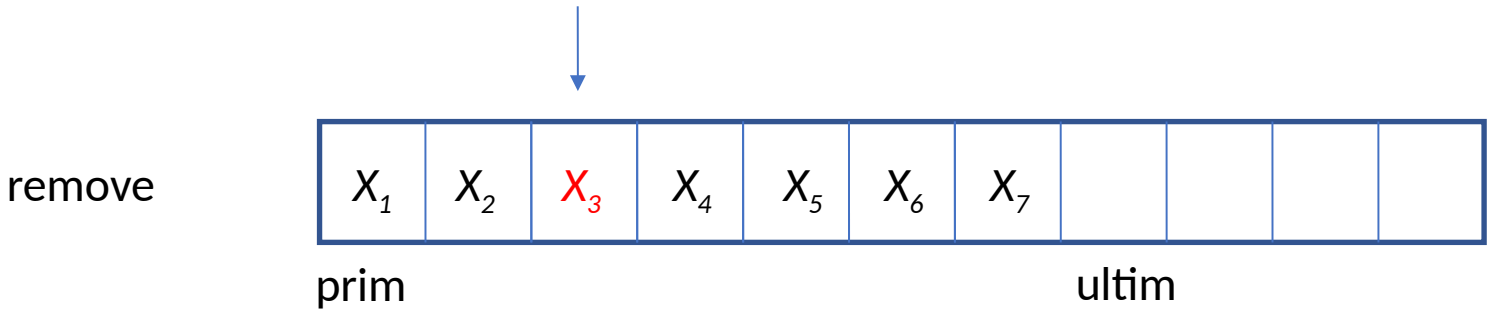
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



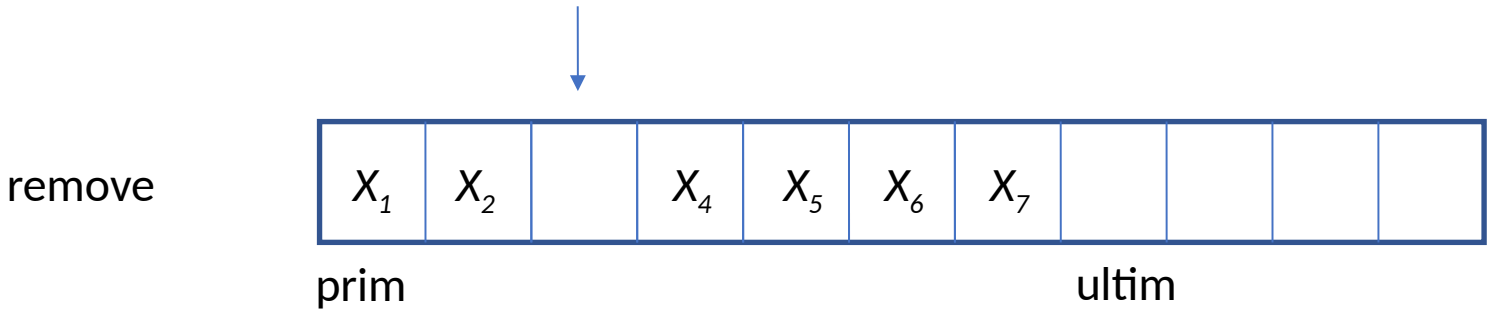
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



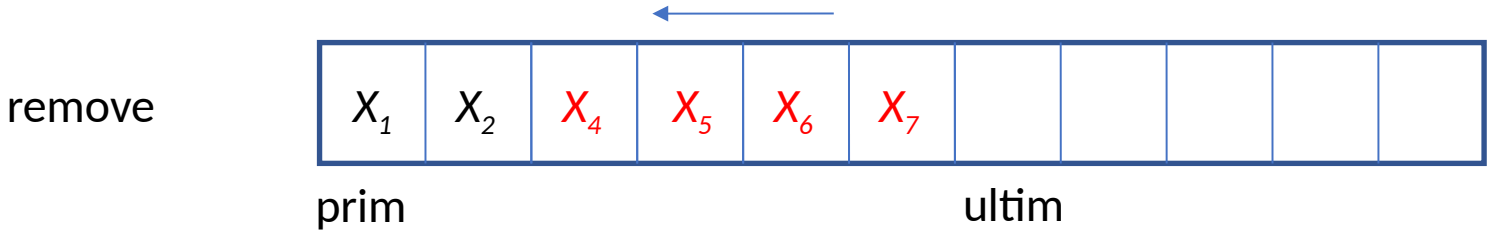
Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



Remover: X_3
Pesquisar até encontrar o elemento.

Propriedade das Listas Lineares



Remover: X_3
Ao encontrar, remover e deslocar todos os demais elementos

Listas por meio de Arranjos

```c
#define MAX 100  // capacidade máxima da lista

typedef struct {
    int dados[MAX];
    int tamanho;
} Lista;

// Criar lista
void criarLista(Lista *l) {
    l->tamanho = 0;
}

// Verificar se está vazia
int estaVazia(Lista *l) {
    return (l->tamanho == 0);
}
```

Listas por meio de Arranjos

```c
// Inserir em qualquer posição
int inserir(Lista *l, int valor, int pos) {
    if (l->tamanho >= MAX) {
        return 0; // lista cheia
    }
    if (pos < 0) pos = 0;
    if (pos > l->tamanho) pos = l->tamanho;

    for (int i = l->tamanho; i > pos; i--) {
        l->dados[i] = l->dados[i - 1];
    }

    l->dados[pos] = valor;
    l->tamanho++;
    return 1;
}
```

Listas por meio de Arranjos

```c
// Buscar elemento (retorna índice ou -1 se não encontrar)
int buscar(Lista *l, int valor) {
    for (int i = 0; i < l->tamanho; i++) {
        if (l->dados[i] == valor) {
            return i;
        }
    }
    return -1;
}

// Remover elemento por valor (retorna 1 se removeu, 0 se não encontrou)
int remover(Lista *l, int valor) {
    int pos = buscar(l, valor);
    if (pos == -1) return 0; // não encontrado

    for (int i = pos; i < l->tamanho - 1; i++) {
        l->dados[i] = l->dados[i + 1];
    }
    l->tamanho--;
    return 1;
}
```

Implementação de Listas por meio de Arranjos

- Vantagem:
  - Economia de memória (os apontadores são implícitos nesta estrutura).
- Desvantagens:
  - Custo para inserir ou retirar itens da lista
    - No pior caso: deslocamento de todos os itens.
  - Em aplicações em que não existe previsão sobre o crescimento da lista
    - Exige a realocação de memória.

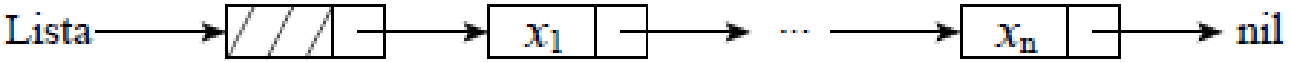
Implementação de Lista por meio de Estruturas Auto Referenciadas

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Também podem ser chamadas de listas encadeadas ou listas ligadas.
- Cada item da lista contém a informação que é necessária para alcançar o próximo item.
- Permite utilizar posições não contíguas de memória.
- Inserir e retirar elementos:
  - Não há necessidade de deslocar os itens seguintes da lista.

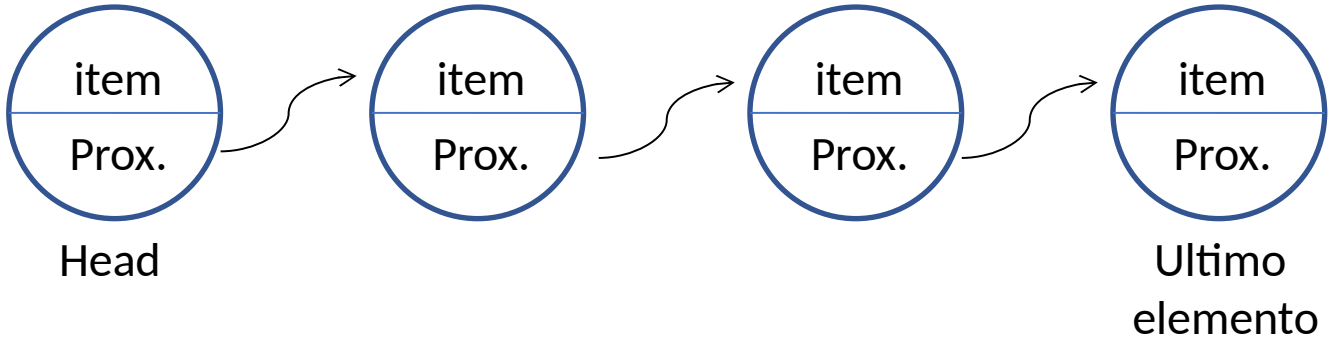
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Há uma célula cabeça (*head*):
  - simplifica as operações sobre a lista



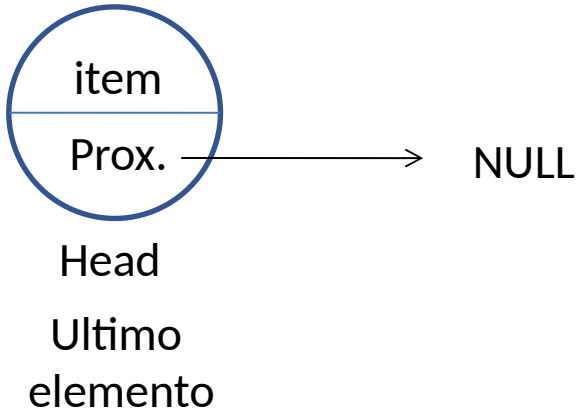
Implementação de Listas por meio de Estruturas Auto Referenciadas

- A lista é constituída de células:
  - Cada uma contém um item da lista e uma referência para a célula seguinte.



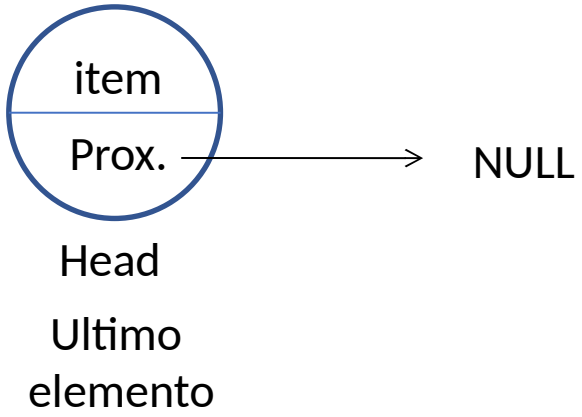
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Construção de uma Lista vazia:



Implementação de Listas por meio de Estruturas Auto Referenciadas

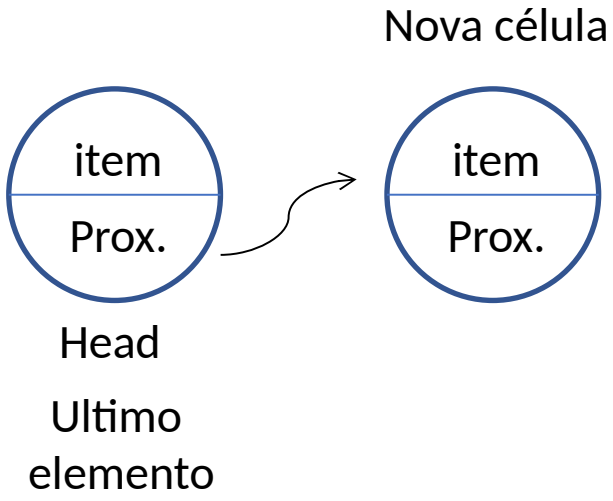
- Inserção de um novo elemento na lista:




```
void insere(Lista *lista, void *x)
{
    Celula *nova = (Celula *)
malloc(sizeof(Celula));
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
    nova->item = x;
    nova->prox = NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

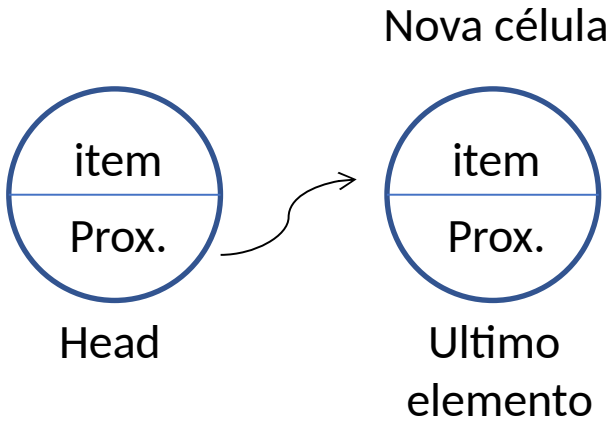
- Inserção de um novo elemento na lista:




```
void insere(Lista *lista, void *x)
{
    Celula *nova = (Celula *)
malloc(sizeof(Celula));
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
    nova->item = x;
    nova->prox = NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

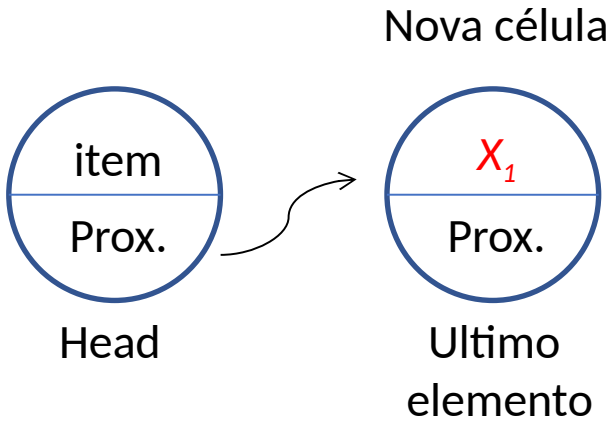
- Inserção de um novo elemento na lista:




```
void insere(Lista *lista, void *x)
{
    Celula *nova = (Celula *)
malloc(sizeof(Celula));
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
    nova->item = x;
    nova->prox = NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

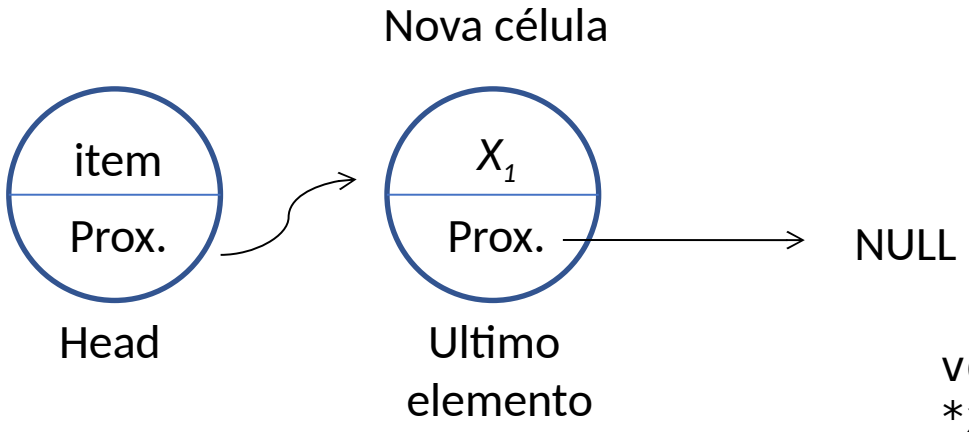
- Inserção de um novo elemento na lista:




```
void insere(Lista *lista, void *x)
{
    Celula *nova = (Celula *)
malloc(sizeof(Celula));
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
    nova->item = x;
    nova->prox = NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

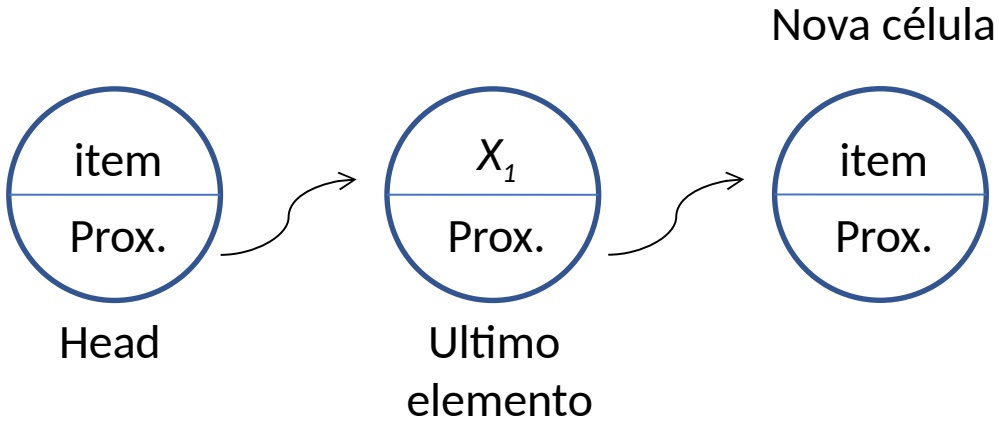
- Inserção de um novo elemento na lista:




```
void insere(Lista *lista, void
*x) {
    Celula *nova = (Celula *)
malloc(sizeof(Celula));
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
    nova->item = x;
    nova->prox = NULL;
}
```

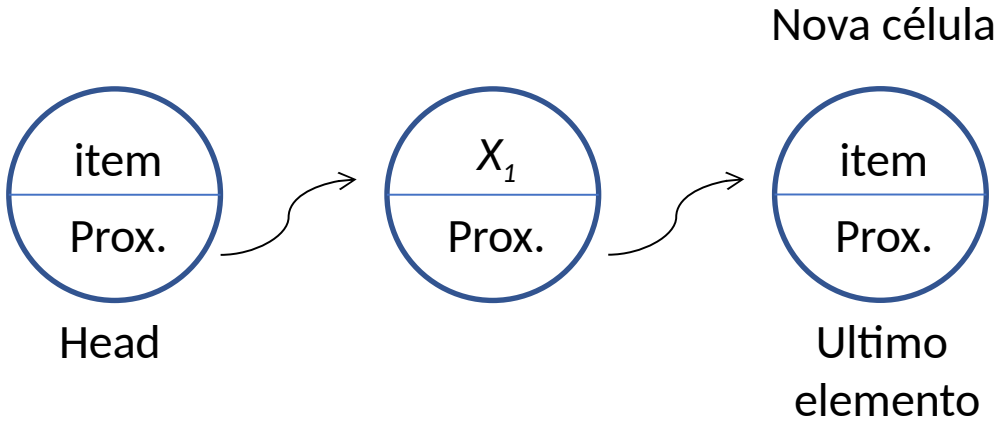
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



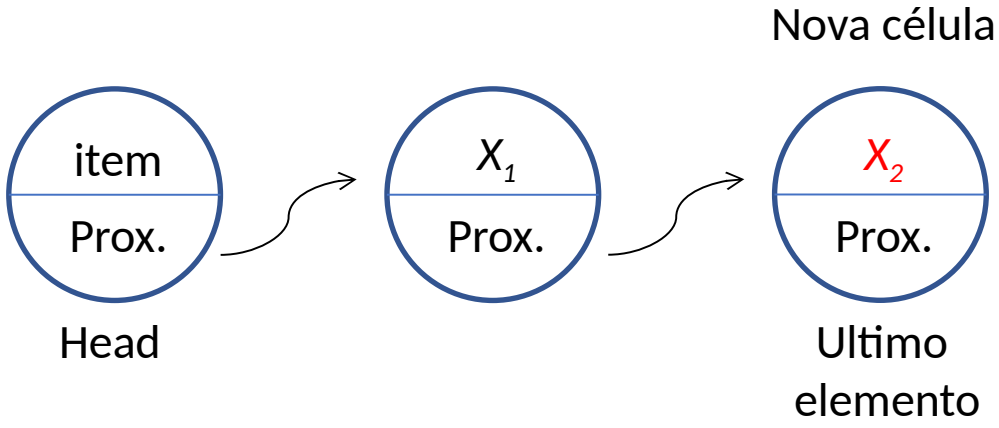
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



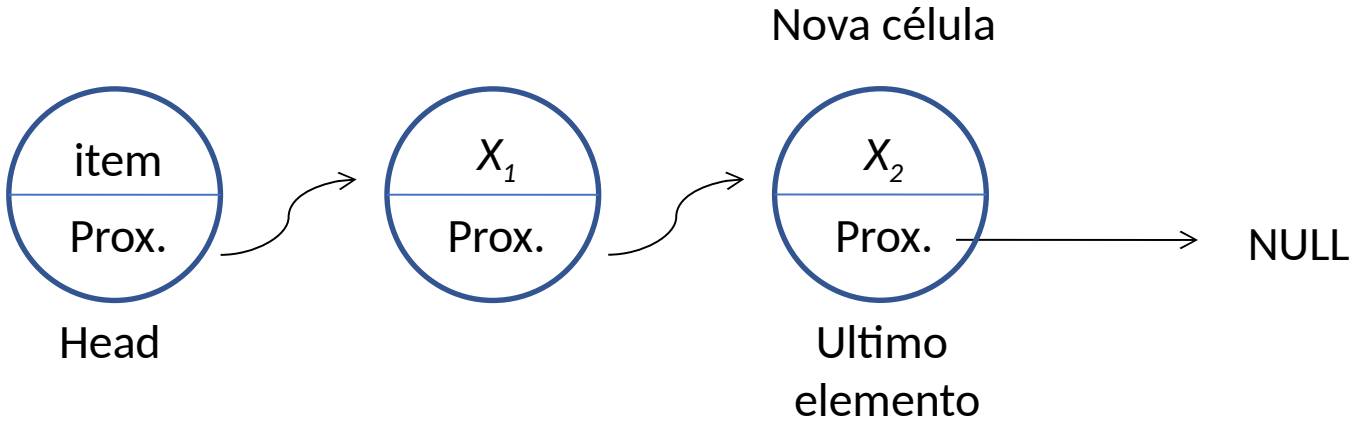
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



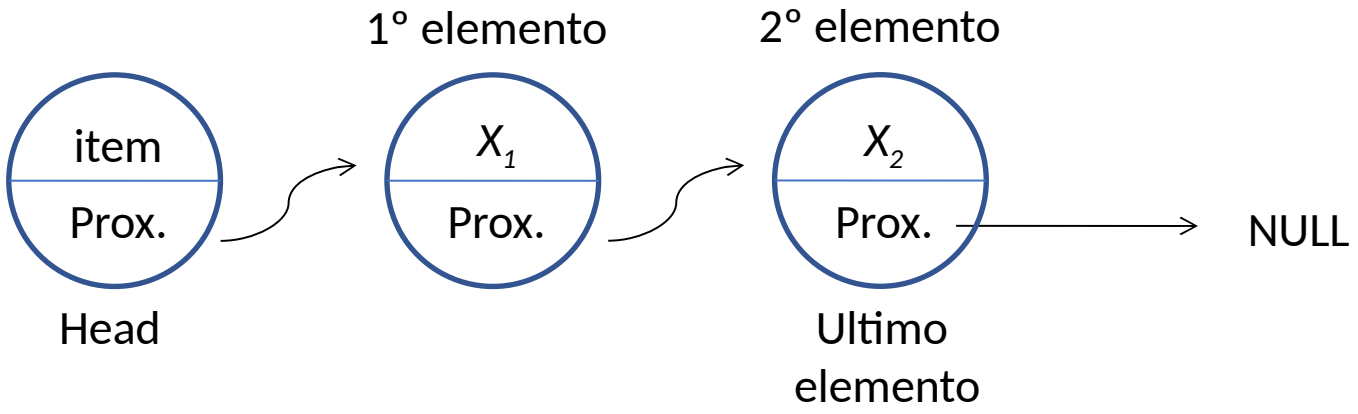
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



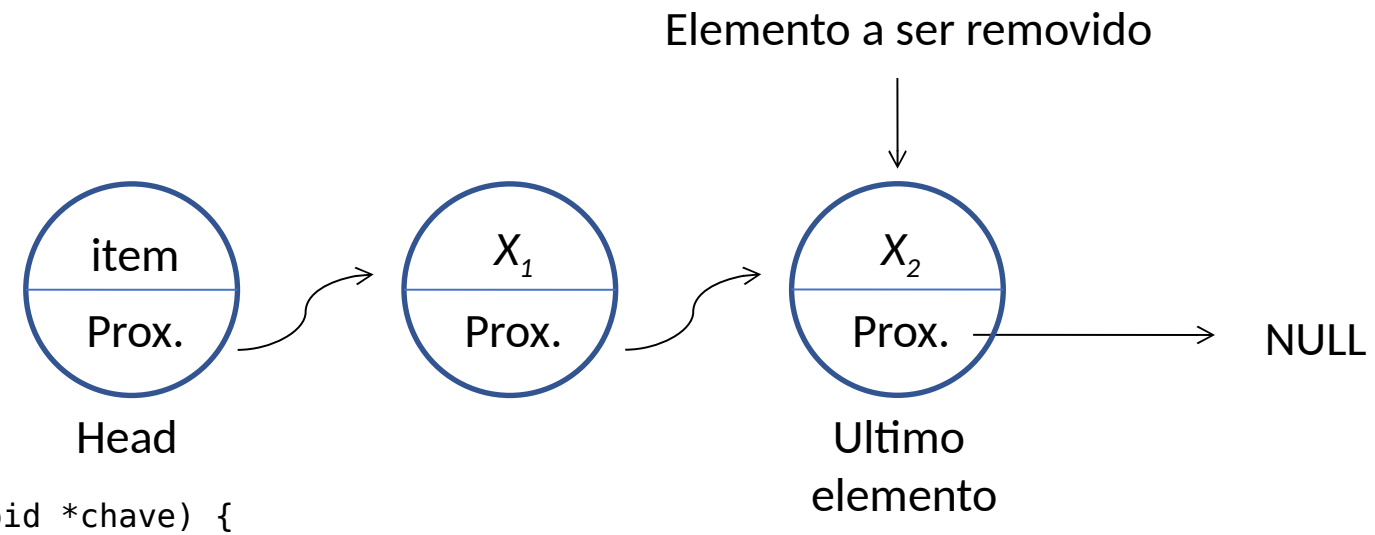
Implementação de Listas por meio de Estruturas Auto Referenciadas

- Inserção de um novo elemento na lista:



Implementação de Listas por meio de Estruturas Auto Referenciadas

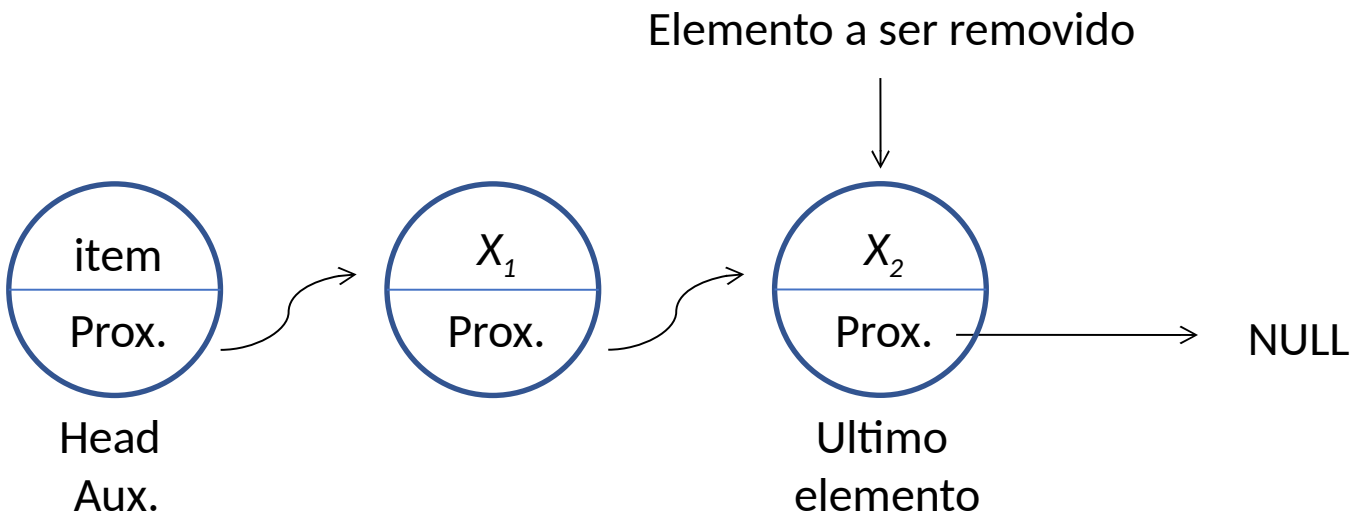
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

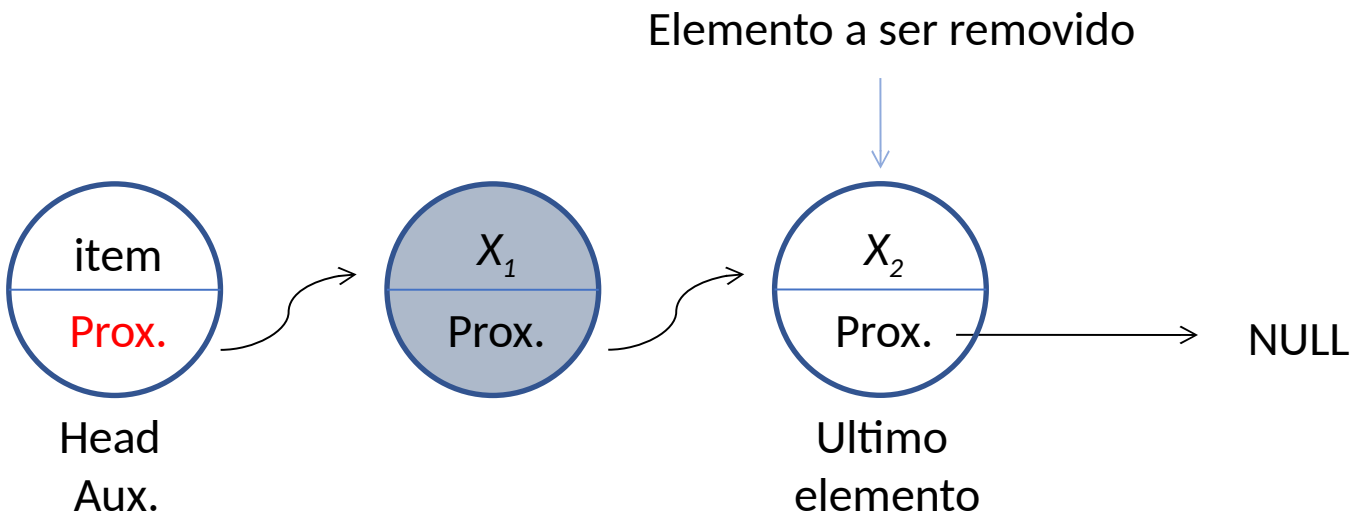



```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Observe que “primeiro” não armazena o primeiro elemento da lista, e sim é uma referência à célula *head* que referencia o primeiro elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

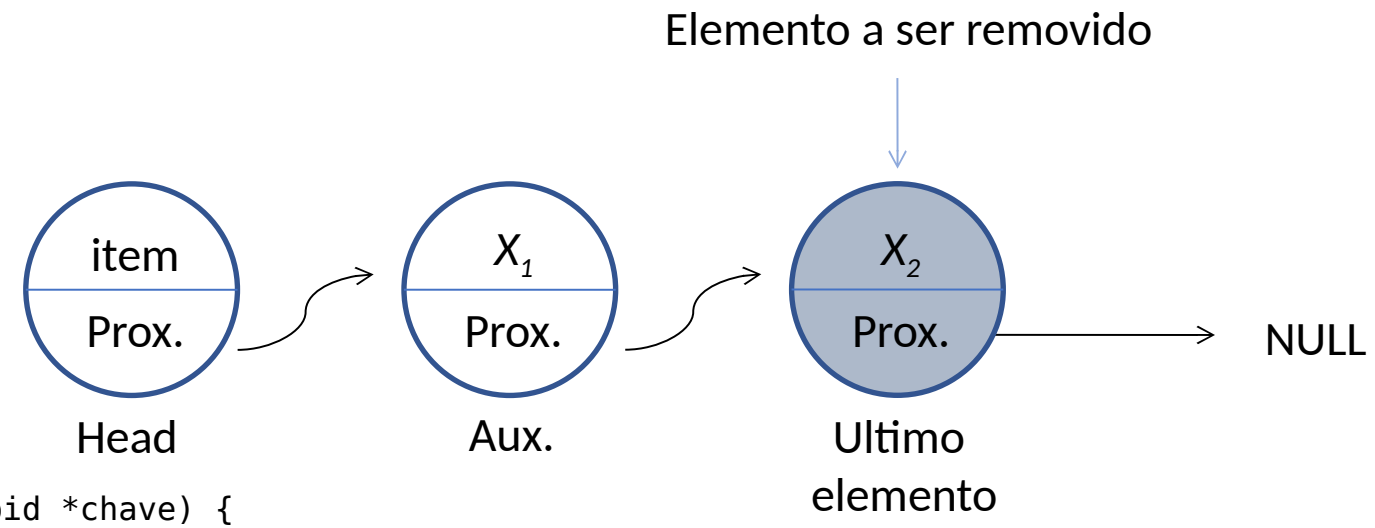
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

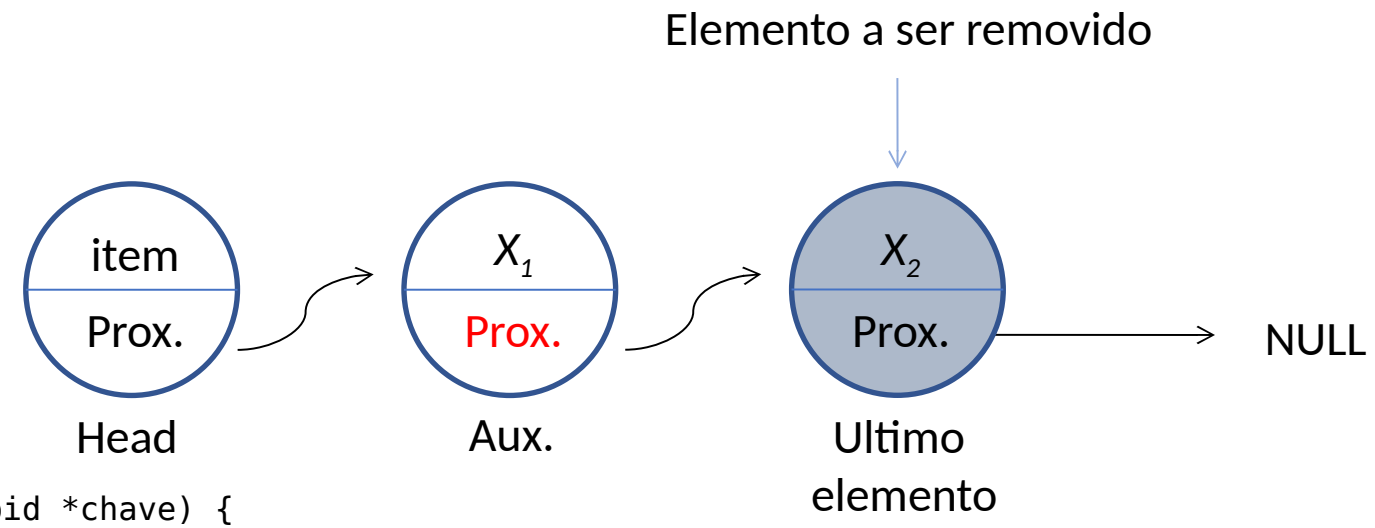
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

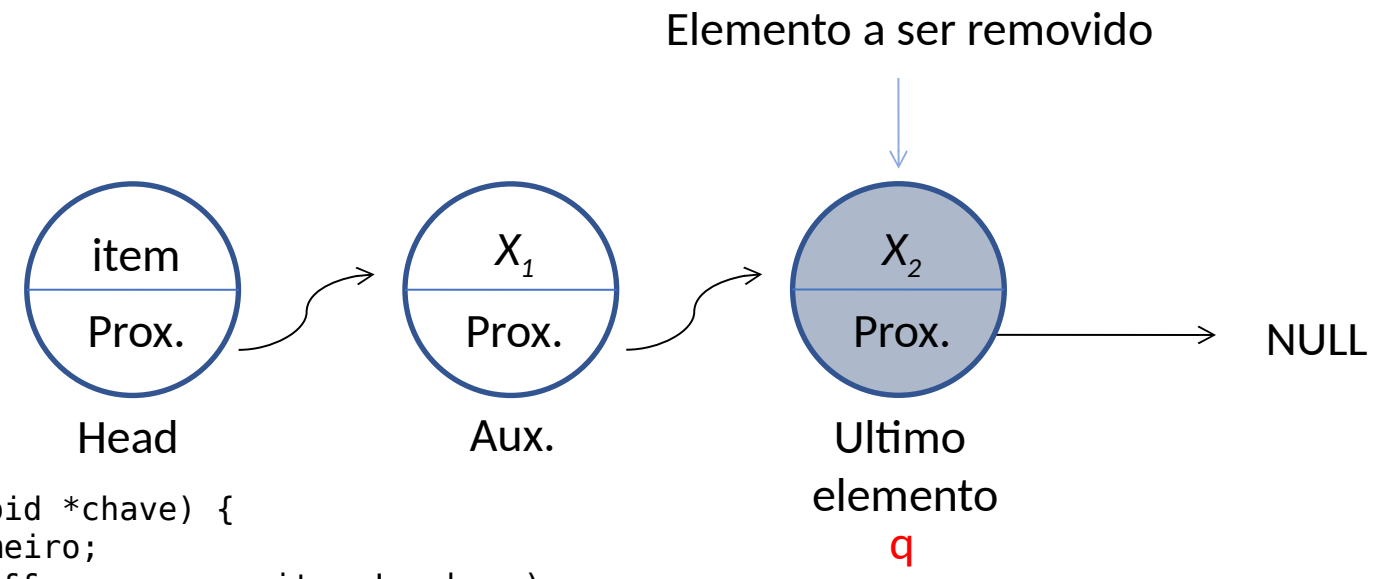
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

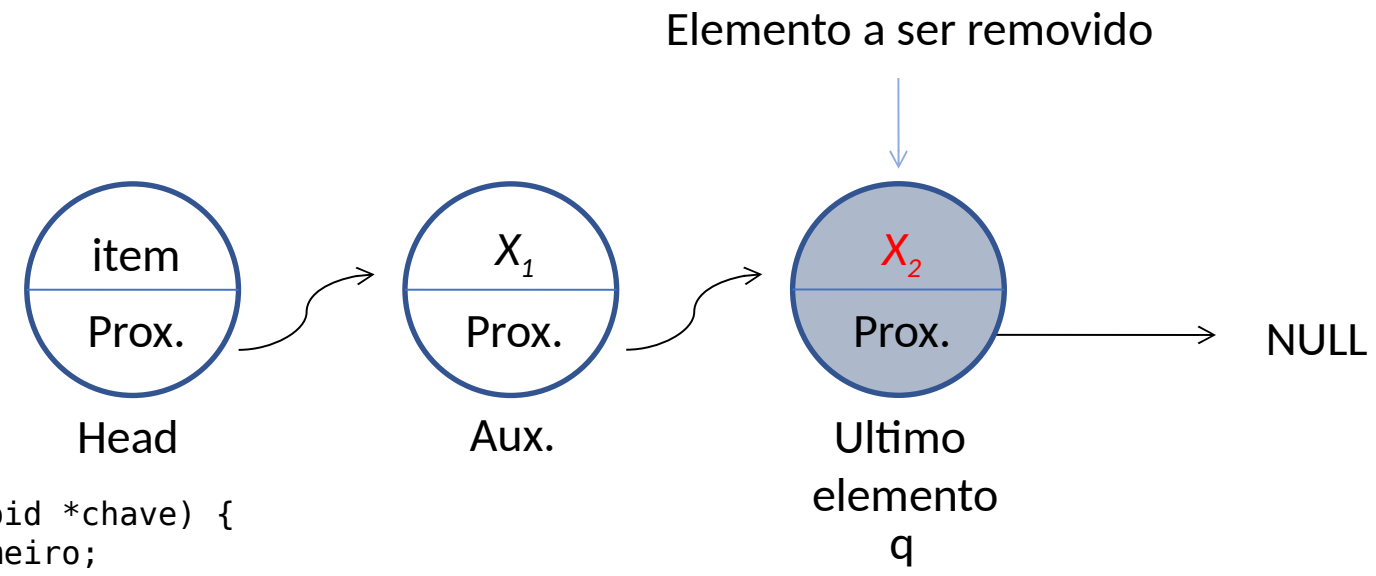
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

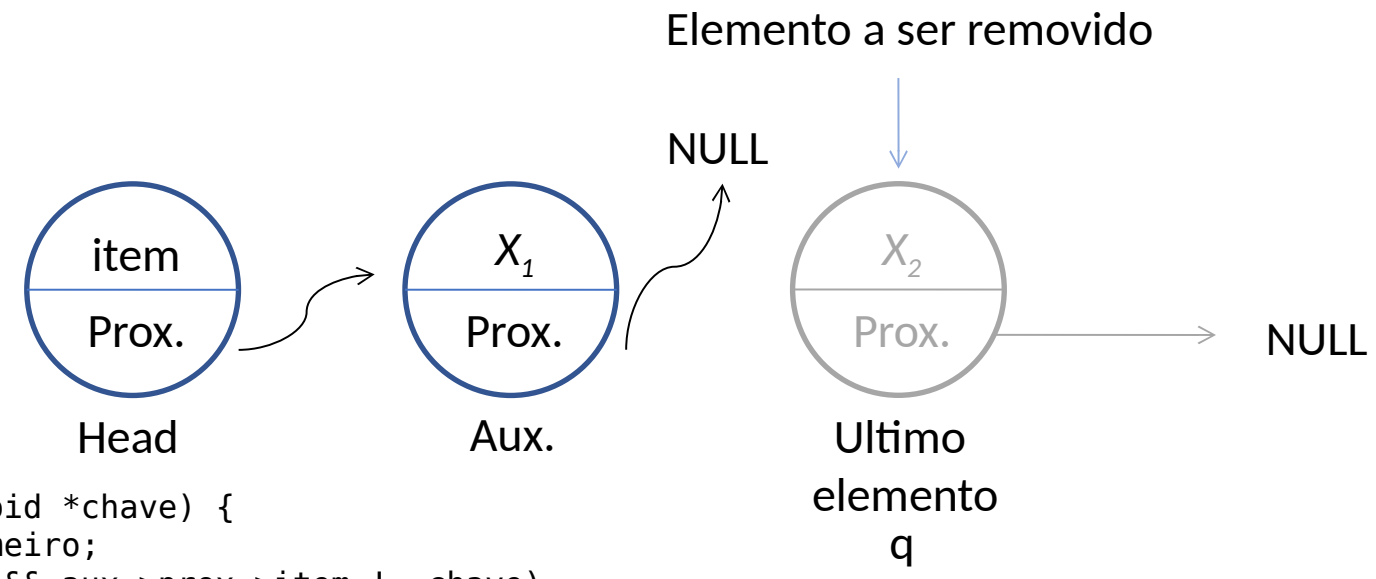



```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Recupera o objeto a ser removido

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

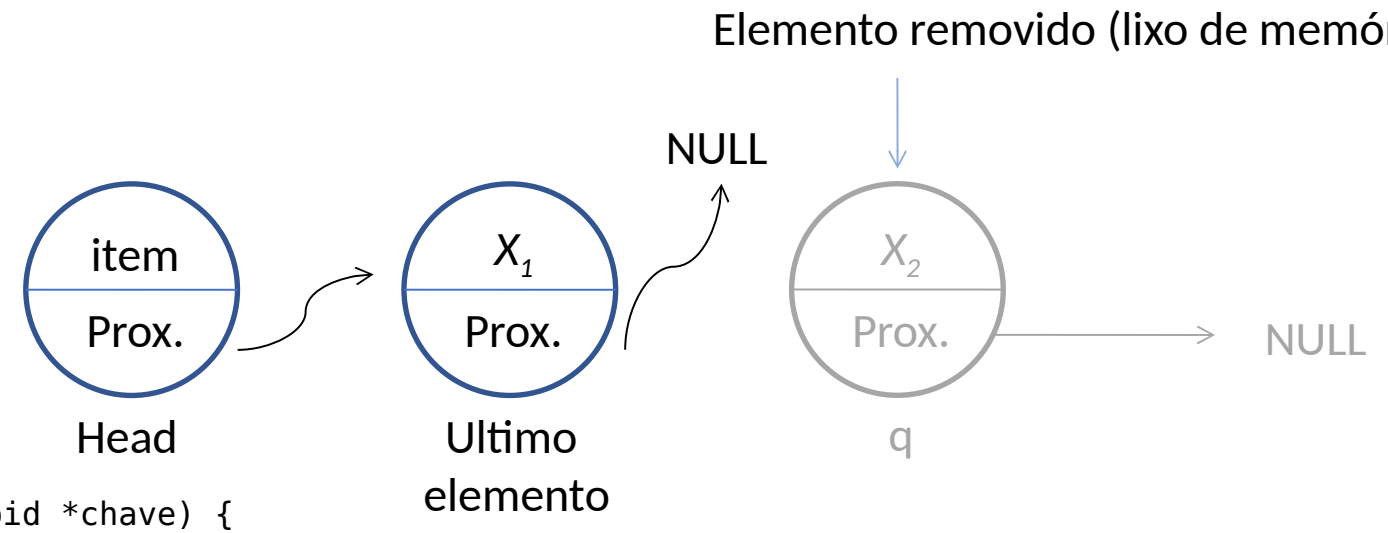



```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Faça aux.prox referenciar o próximo elemento da lista. Neste caso nulo, pois não há mais elementos.

Implementação de Listas por meio de Estruturas Auto Referenciadas

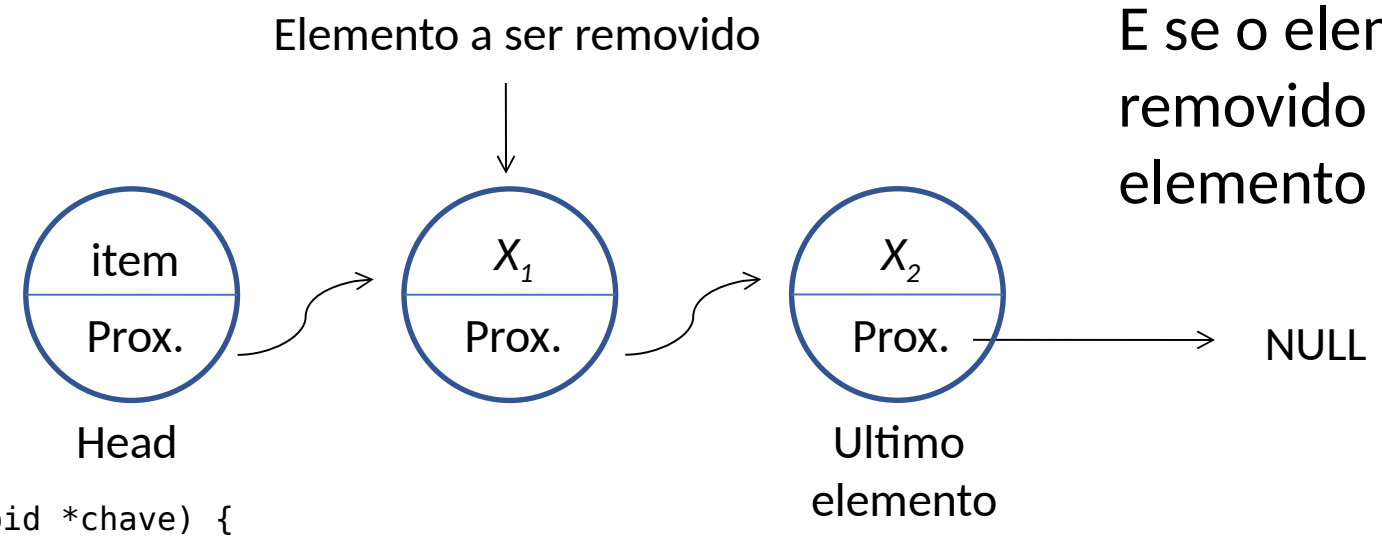
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

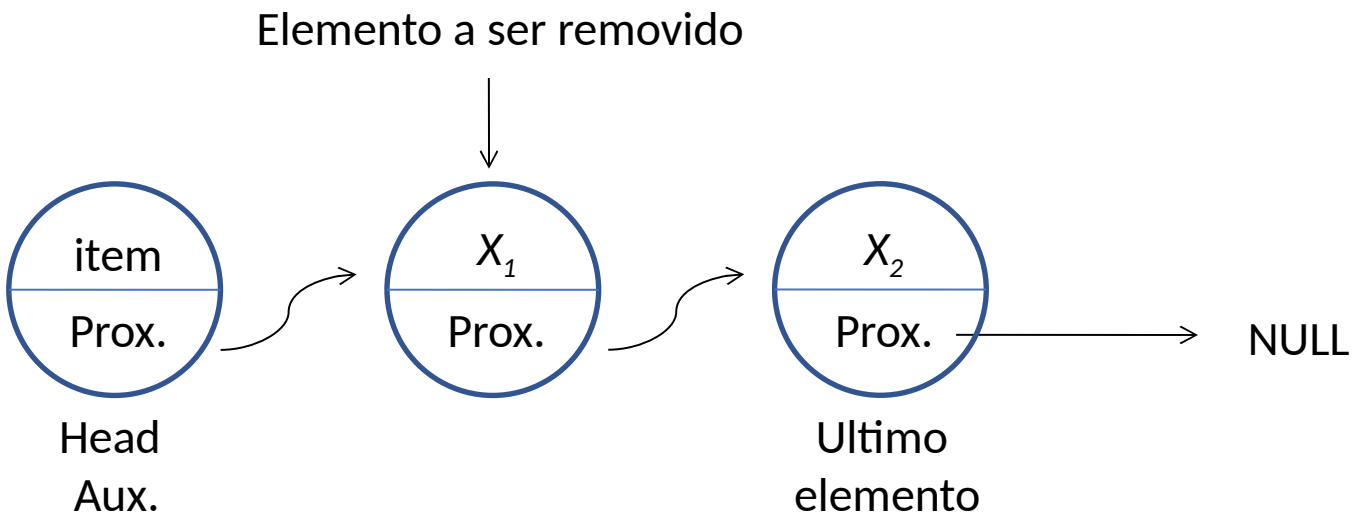


E se o elemento a ser removido não for o último elemento da lista?

```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

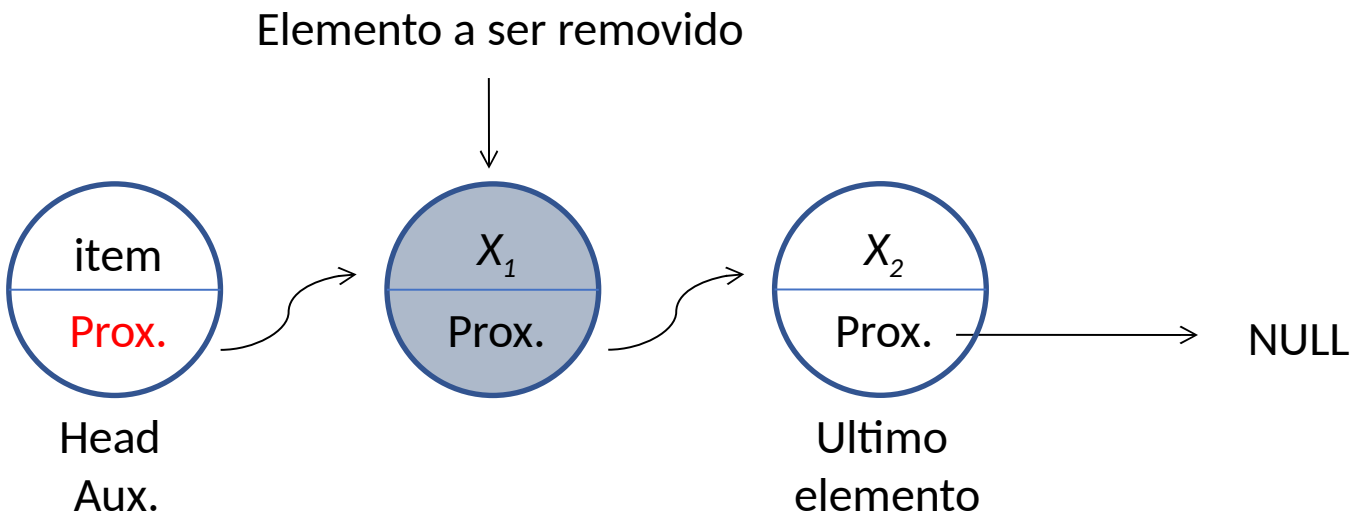



```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Observe que o objeto “primeiro” não armazena o primeiro elemento da lista, e sim é uma referência à célula *head* que referencia o primeiro elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

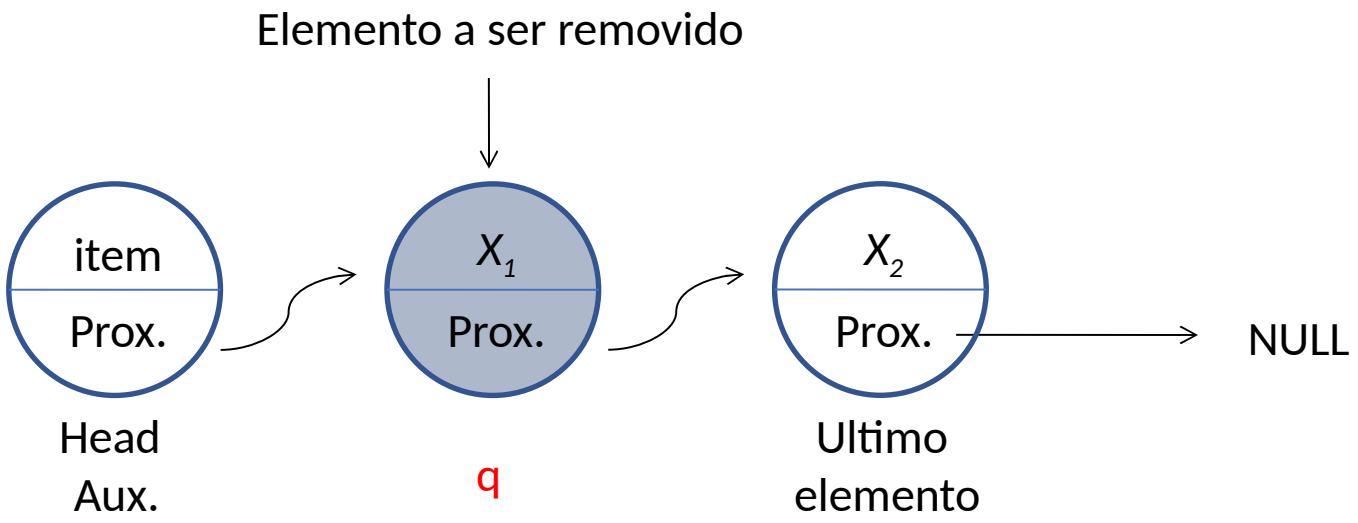
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

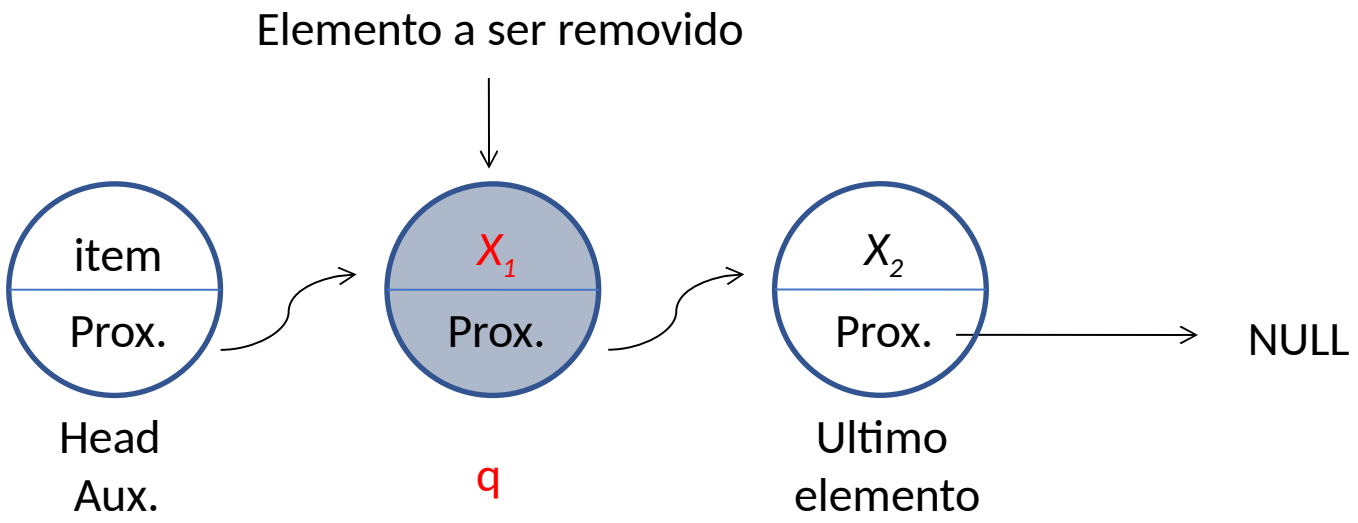
- Remoção de um elemento na lista:




```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:

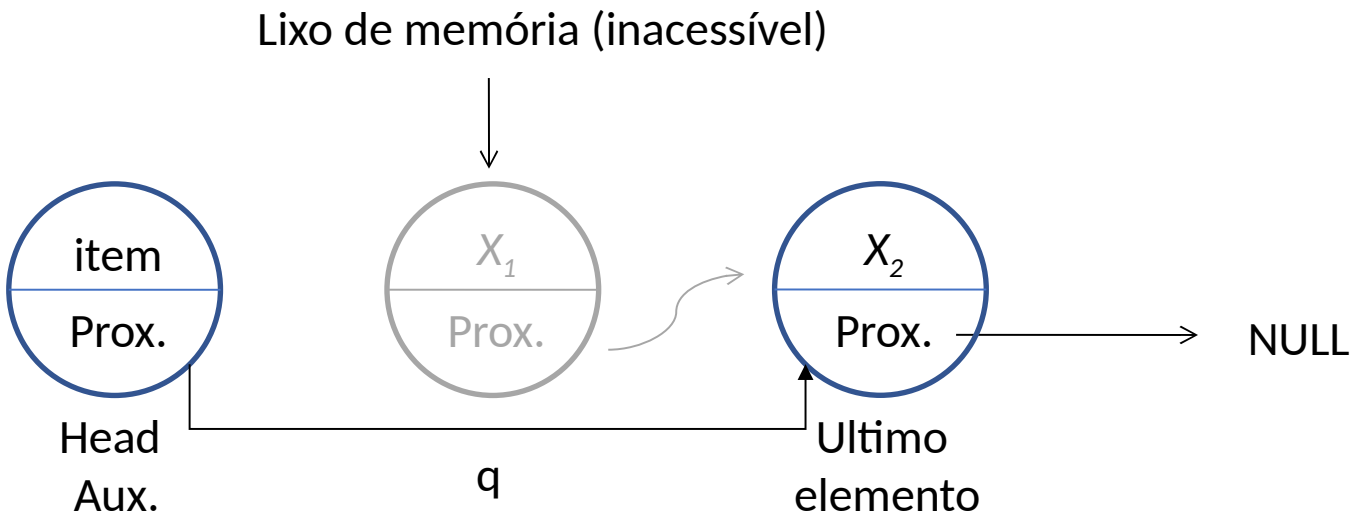



```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Recupera o objeto a ser removido

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Remoção de um elemento na lista:



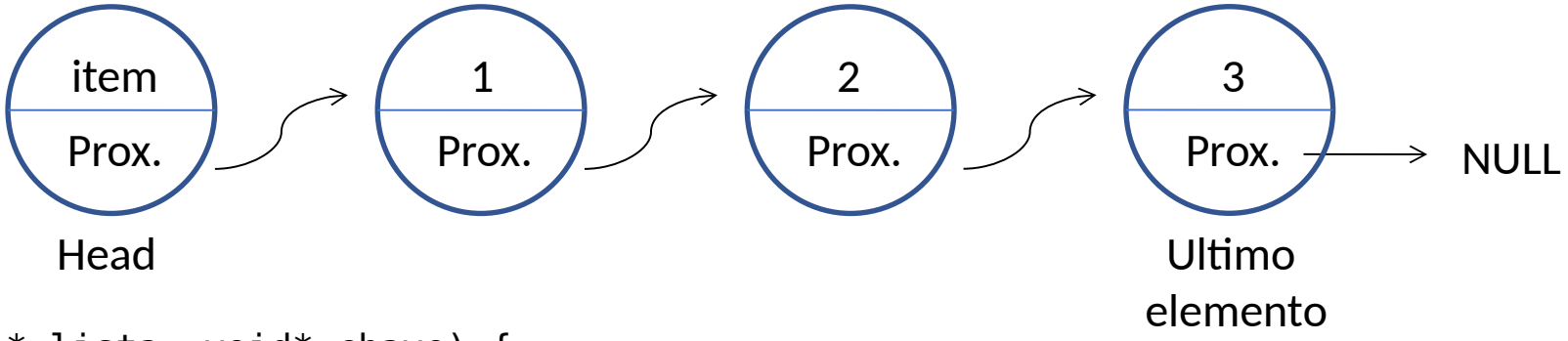

```
void* retira(Lista *lista, void *chave) {
    Celula *aux = lista->primeiro;
    while (aux->prox != NULL && aux->prox->item != chave)
        aux = aux->prox;
    if (aux->prox == NULL) return NULL;
    Celula *q = aux->prox; void *item = q->item;
    aux->prox = q->prox;
    if (aux->prox == NULL) lista->ultimo = aux;
    free(q);
    return item;
}
```

Faça aux.prox referenciar o próximo elemento da lista. Neste caso, o último elemento.

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

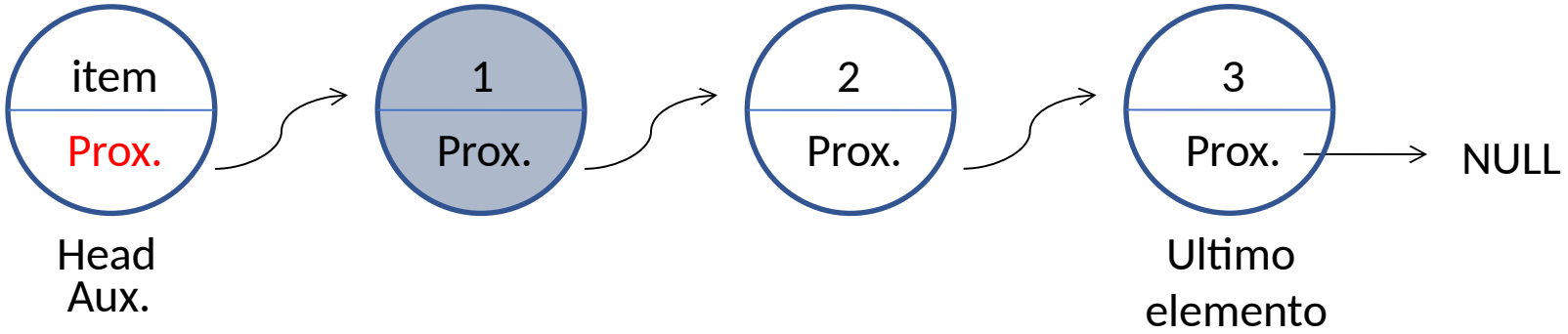



```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

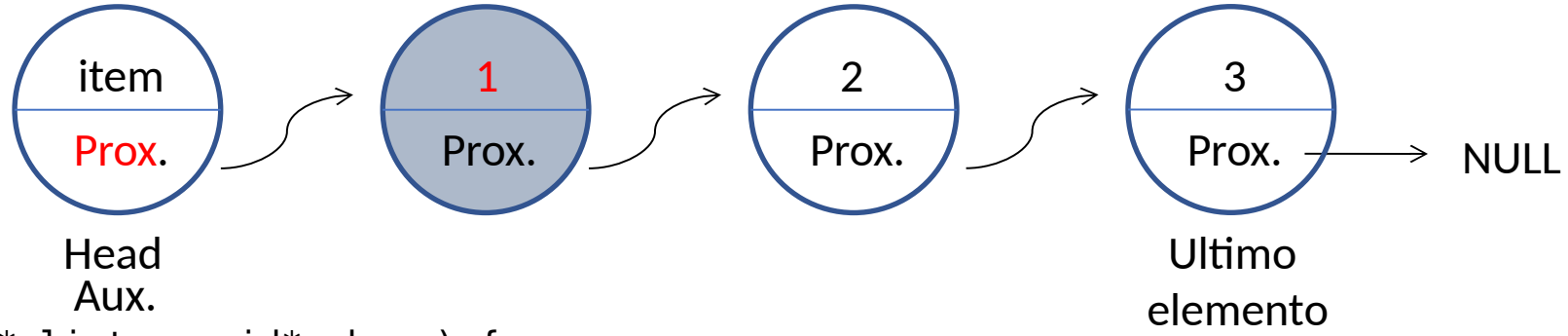



```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

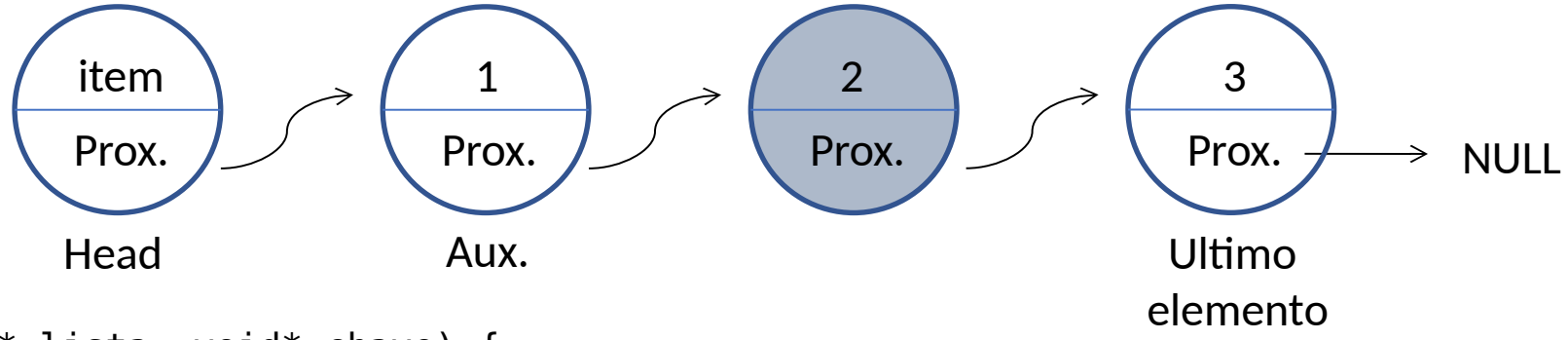



```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

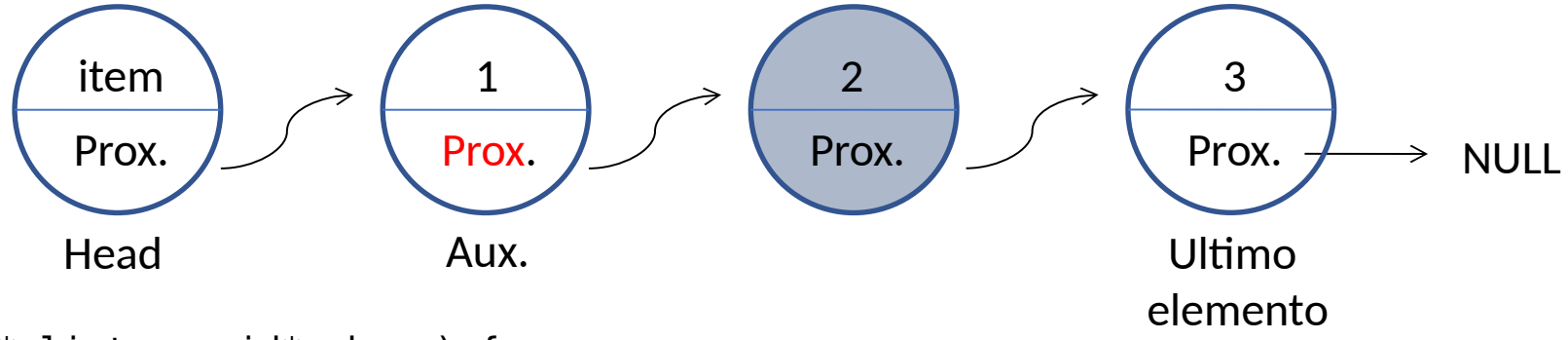



```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);

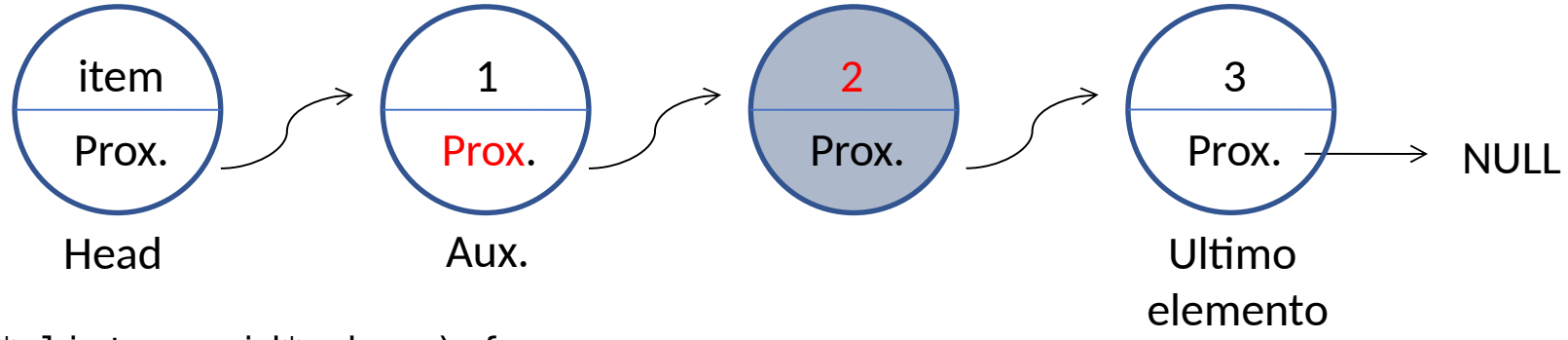



```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Pesquisa de um elemento na lista:

pesquisa(2);




```
void* pesquisa(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro;
    while (aux->prox != NULL) {
        if (aux->prox->item == chave)
            return aux->prox->item;
        aux = aux->prox;
    }
    return NULL;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- A classe Lista contém:
  - uma referência para a célula *head*;
  - uma referência para a última célula da lista; e
  - uma referência para armazenar a posição corrente na lista.

Filas usando Estruturas Auto Referenciadas

```c
#include <stdio.h>
#include <stdlib.h>

// Definição das estruturas
typedef struct Celula {
    void* item;
    struct Celula* prox;
} Celula;

typedef struct {
    Celula* primeiro;  // sentinela
    Celula* ultimo;
} Lista;

// Cria uma nova lista com sentinela
Lista* criarLista() {
    Lista* lista = malloc(sizeof(Lista));
    lista->primeiro = malloc(sizeof(Celula));
    lista->primeiro->item = NULL;
    lista->primeiro->prox = NULL;
    lista->ultimo = lista->primeiro;
    return lista;
}
```

Filas usando Estruturas Auto Referenciadas

```c
void destruirLista(Lista* lista) {
    Celula* atual = lista->primeiro;
    while (atual != NULL) {
        Celula* temp = atual;
        atual = atual->prox;
        free(temp);
    }
    free(lista);
}

// Verifica se a lista está vazia
int estaVazia(Lista* lista) {
    return lista->primeiro->prox == NULL;
}

// Insere um novo item no final da lista
void inserir(Lista* lista, void* item) {
    Celula* nova = malloc(sizeof(Celula));
    nova->item = item;
    nova->prox = NULL;
    lista->ultimo->prox = nova;
    lista->ultimo = nova;
}
```

Filas usando Estruturas Auto Referenciadas

```c
int remover(Lista* lista, void* chave) {
    Celula* anterior = lista->primeiro;
    while (anterior->prox != NULL) {
        if (anterior->prox->item == chave) {
            Celula* temp = anterior->prox;
            anterior->prox = temp->prox;
            if (temp == lista->ultimo) {
                lista->ultimo = anterior;
            }
            free(temp);
            return 1; // sucesso
        }
        anterior = anterior->prox;
    }
    return 0; // não encontrado
}
```

Filas usando Estruturas Auto Referenciadas

```c
void* buscar(Lista* lista, void* chave) {
    Celula* aux = lista->primeiro->prox;
    while (aux != NULL) {
        if (aux->item == chave) {
            return aux->item;
        }
        aux = aux->prox;
    }
    return NULL;
}

// Retorna o próximo item de uma célula
void* proximo(Celula* celula) {
    if (celula == NULL || celula->prox == NULL) return NULL;
    return celula->prox->item;
}

// Retorna a primeira célula (sentinela)
Celula* primeiraCelula(Lista* lista) {
    return lista->primeiro;
}
```

Implementação de Listas por meio de Estruturas Auto Referenciadas

- Vantagem:
  - Permite inserir ou retirar itens do meio da lista a um custo constante
    - importante para lista de elementos ordenados.
  - Bom para aplicações em que não existe previsão sobre o crescimento da lista
    - o tamanho máximo da lista não precisa ser definido.

- Desvantagens:
  - utilização de memória extra para armazenar as referências

Exercícios

- Implementar classe Lista (por vetor) e Lista (encadeada), e implementar, para cada tipo de lista, um programa que:
  - Construa a lista vazia;
  - Insira novos elementos na lista;
  - Pesquisa um elemento;
  - Retire um elemento.
- Para a Lista (por vetor), implemente uma função para realocar o tamanho do vetor de itens da lista.
- Implemente uma função para Lista (Lista encadeada) para inserir um elemento em qualquer posição da lista ordenada.
  - Crie a lista com os elementos: 1, 2, 3, 5, 6.
  - Insira o elemento 4 de forma ordenada na lista.

Listas Duplamente Ligadas

- As Operações de remoção de uma elemento indica um problema inerente às listas ligadas.

- As células das listas contém apenas a referência para os seus sucessores.

- Não existe um acesso imediato aos seus predecessores.

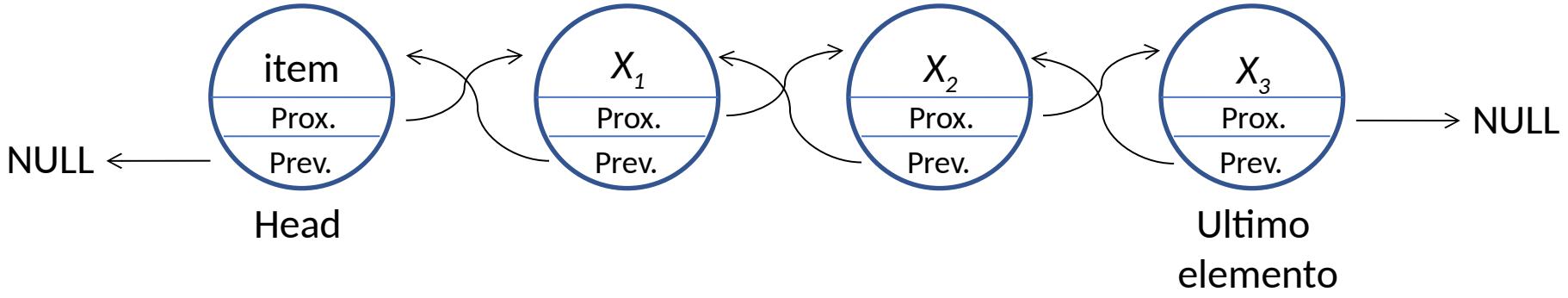
Listas Duplamente Ligadas

- Para remover o último elemento da lista deve-se percorrer toda a lista até encontrar o penúltimo elemento.

- Para listas longas e muita utilização da remoção do ultimo elemento, podem impactar no processamento rápido da lista.

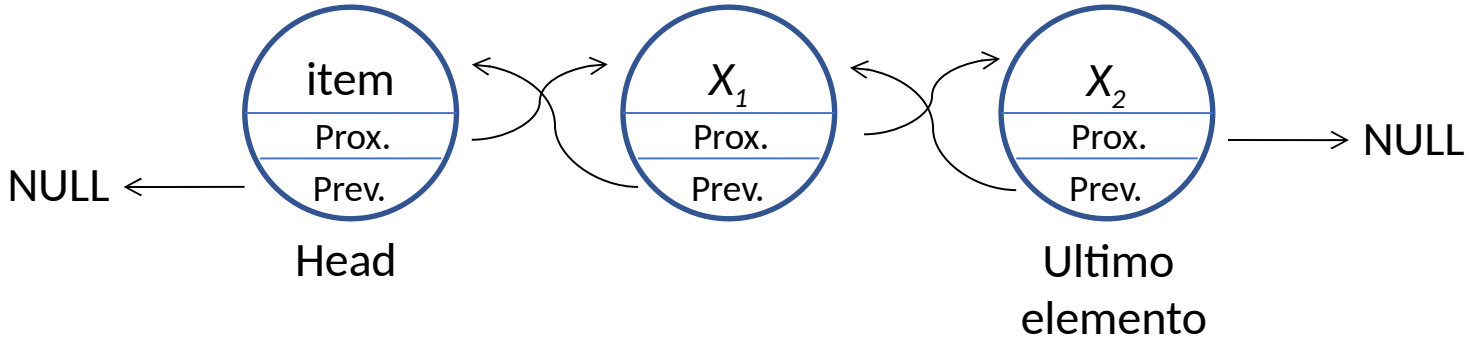
Listas Duplamente Ligadas

- As listas duplamente ligadas (ou duplamente encadeadas) cada célula da lista contem duas referências:
  - Uma para o sucessor (igual nas listas ligadas)
  - Outra para o predecessor.



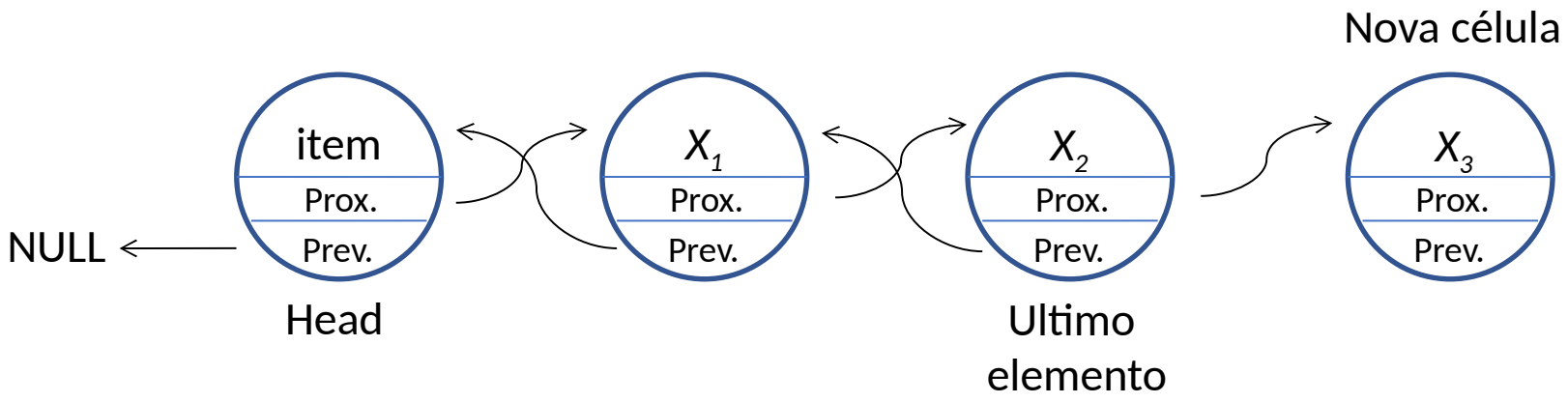
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - Lista com dois elementos:



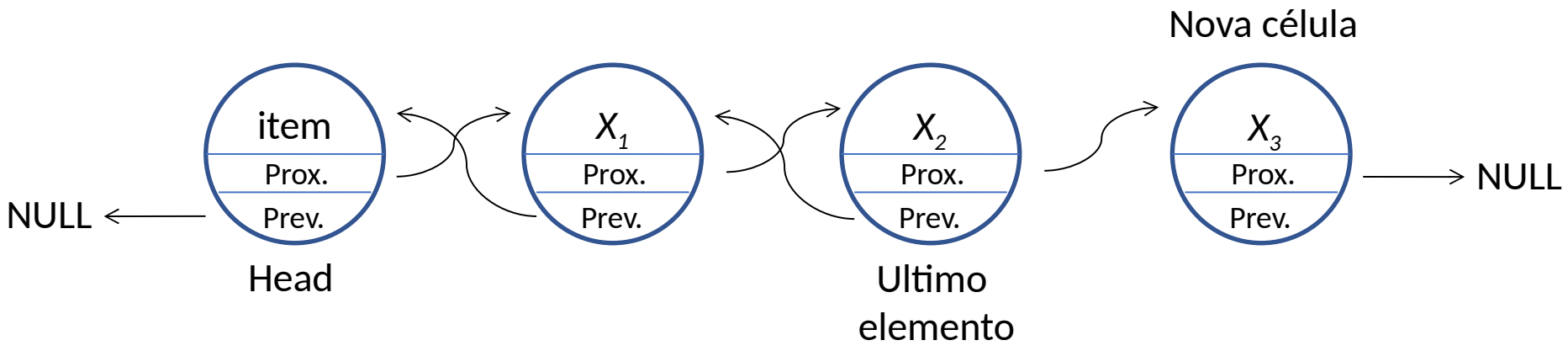
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - Uma nova célula é criada e seus três atributos são inicializados:



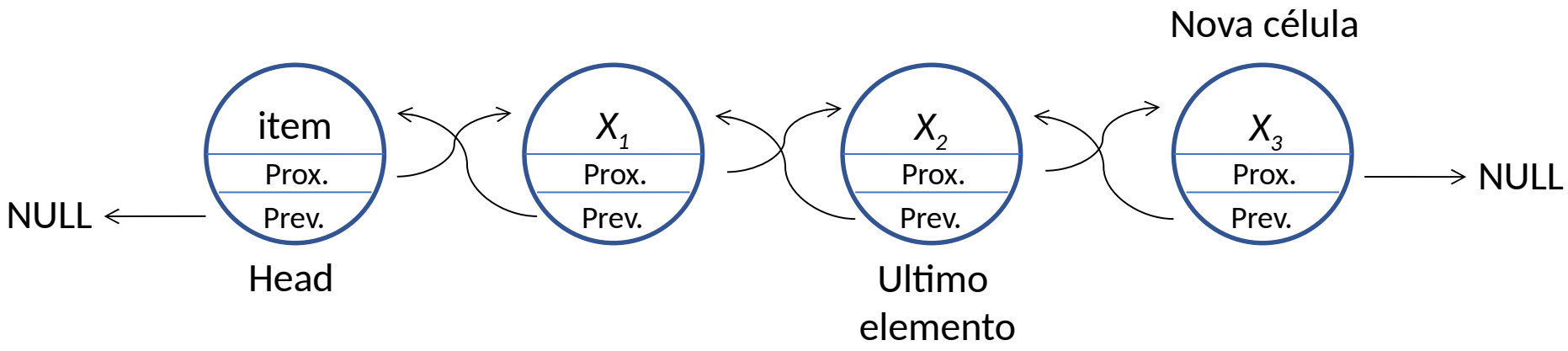
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “prox” da nova célula referencia nulo:



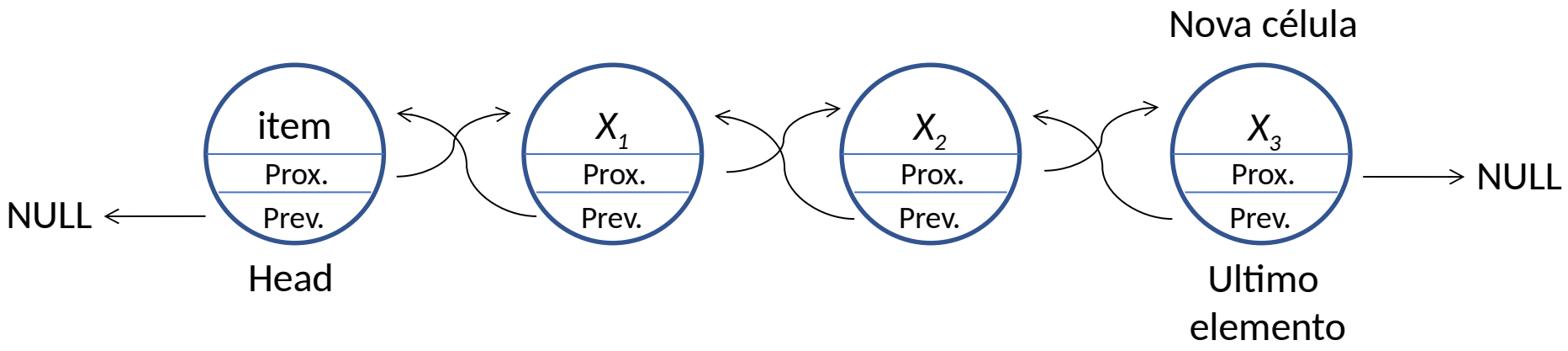
Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “prev” referencia o (até então) último elemento da lista:



Listas Duplamente Ligadas

- Inserção de um novo elemento a lista:
  - “ultimo” é ajustado para o novo elemento inserido:

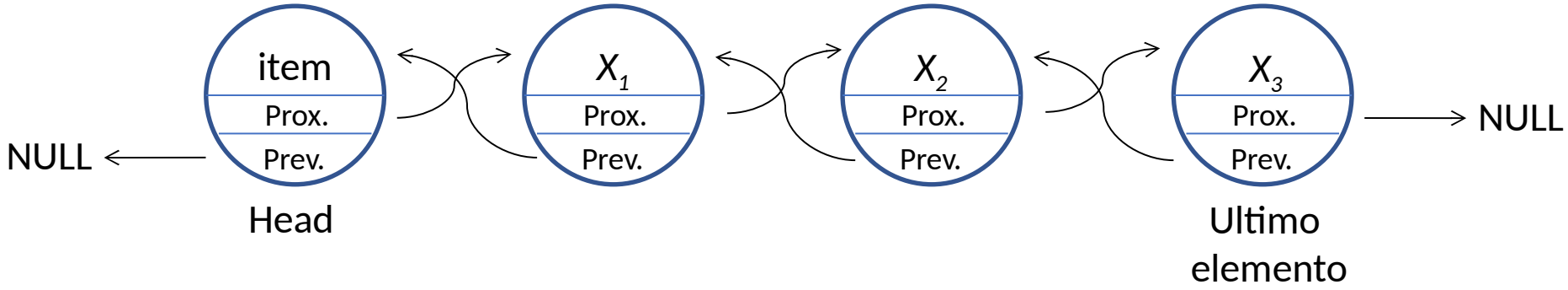


Listas Duplamente Ligadas

- Remoção do último elemento da lista:
  - Remover o último elemento de uma lista duplamente ligada é simples pois há acesso direto do ultimo elemento ao seu predecessor, e nenhum laço é necessário para remover o último elemento

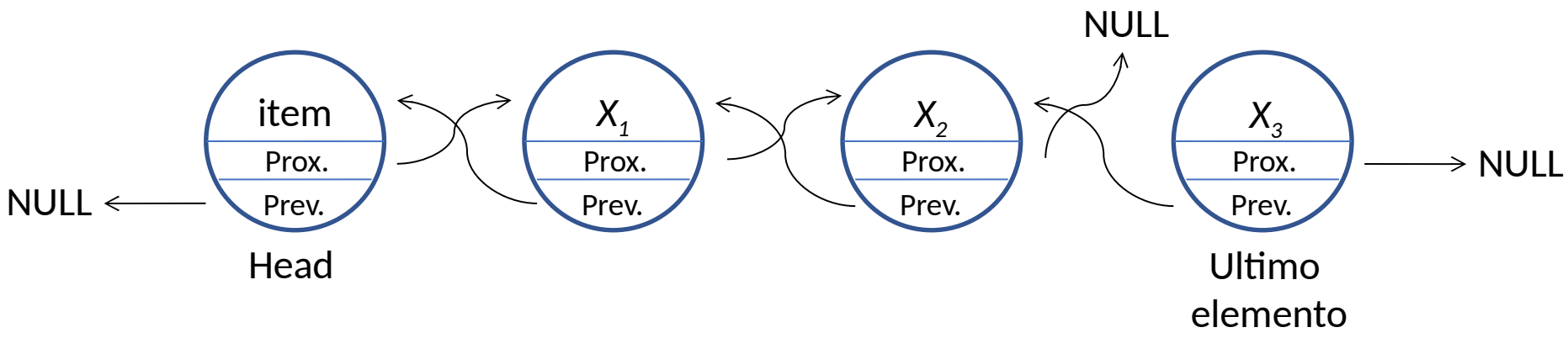
Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Não é necessário percorrer a lista.



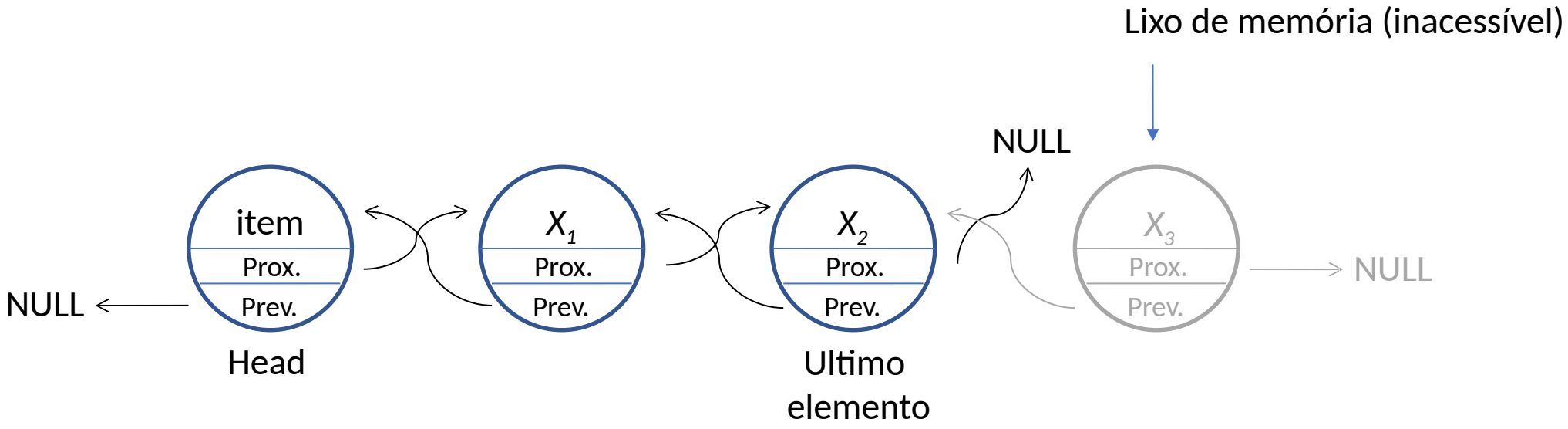
Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Após recuperar o item a ser removido, “prox” do predecessor passa a referenciar NULL



Listas Duplamente Ligadas

- Remoção do ultimo elemento da lista:
  - Ultimo elemento é ajustado para o predecessor do elemento a ser removido.



Listas Duplamente Ligadas

- Devido a imediata acessibilidade ao ultimo elemento, tanto a operação de inserção quanto a remoção do ultimo elemento são executadas sem a necessidade de percorrer a lista.

Listas Circulares

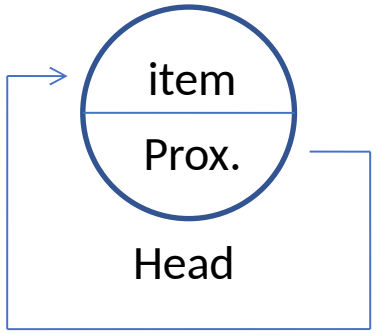
- São listas na qual as células formam um anel:
  - A lista é finita; e
  - Cada célula tem um sucessor.
- O sucessor do último elemento é o primeiro elemento da lista.
- Uma mudança na lista linear:
  - O campo *prox* do ultimo elemento referencia o primeiro elemento, ou invés de nulo.

Listas Circulares

- Neste caso, pode-se usar somente uma referência permanente: o primeiro elemento da lista, ao contrário da lista ligada que utilizava duas: primeiro (*head*) e ultimo.

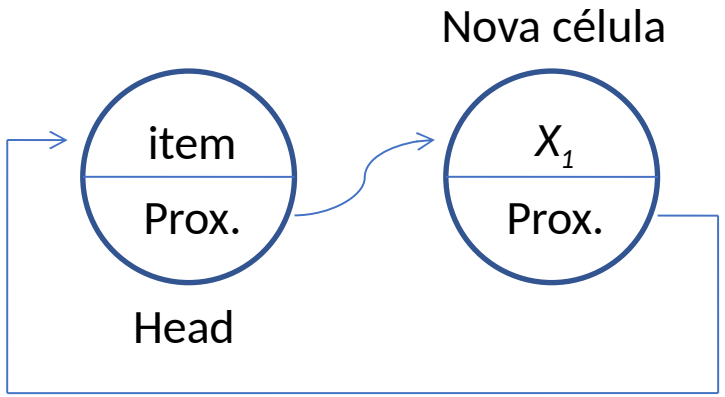
Listas Circulares

- Lista circular vazia (note que não há mais a referência ao ultimo elemento):



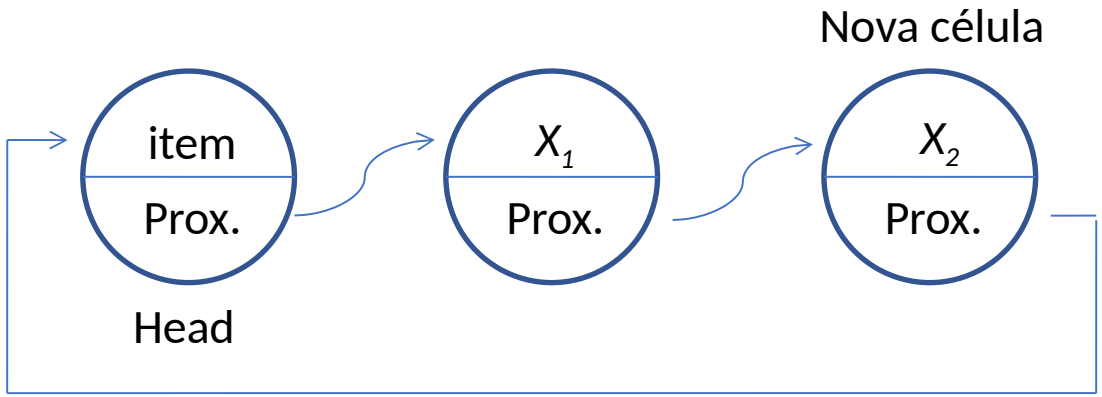
Listas Circulares

- Inserção de um novo elemento na lista.



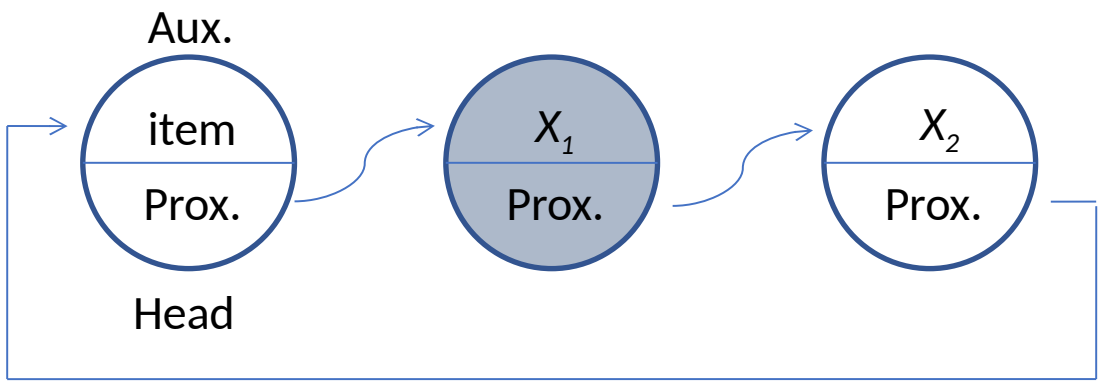
Listas Circulares

- Inserção de um novo elemento na lista.



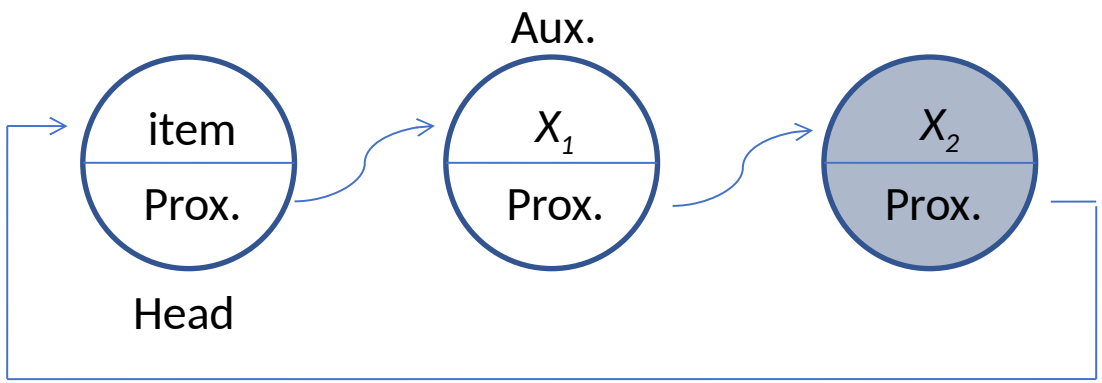
Listas Circulares

- Removendo o ultimo elemento da lista.



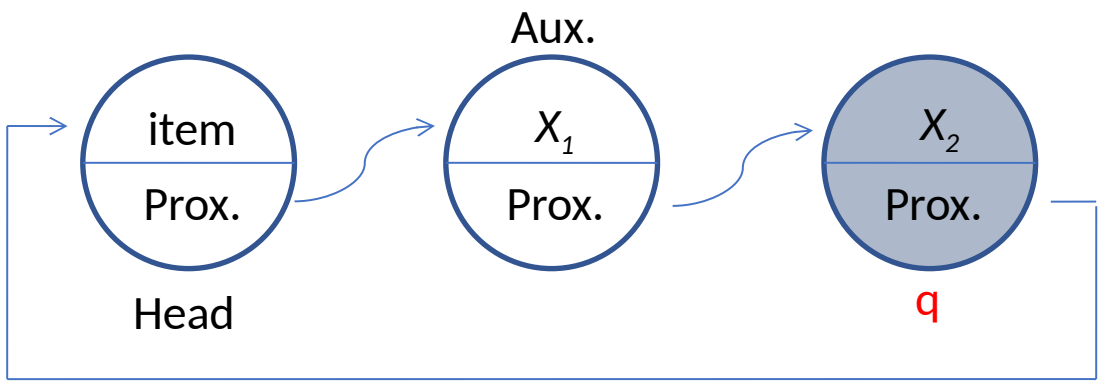
Listas Circulares

- Removendo o ultimo elemento da lista.



Listas Circulares

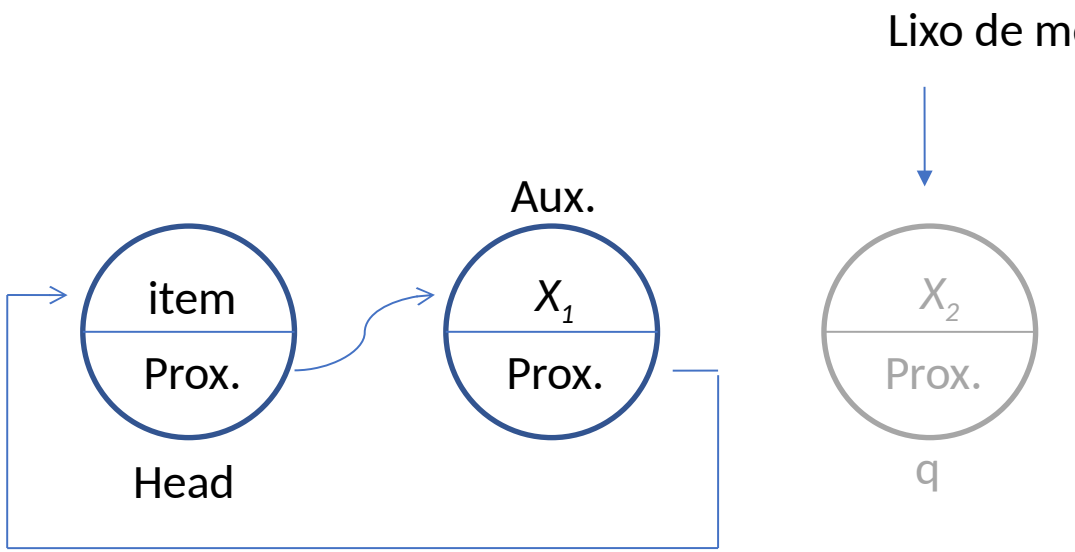
- Removendo um elemento da lista.



Recupera o elemento
a ser removido

Listas Circulares

- Removendo um elemento da lista.



Ao invés de referenciar nulo, passa a
referenciar o primeiro elemento (próximo elemento).

Listas Circulares

- Como nas listas lineares, para remover o ultimo elemento é necessário percorrer toda a lista.

- No entanto, com listas circulares é possível acessar qualquer elemento de qualquer ponto da lista (uma vez que a partir do último é possível acessar o primeiro).

Exercícios

- Baseado na implementação da lista ligada (lista auto referenciada ou encadeada) apresentada em aula, implemente as listas Duplamente Encadeada e a Lista Circular e suas operações básicas.
- Para a lista Duplamente Encadeada implemente operações que:
  - Insere novo elemento no meio da lista (ordenada);
  - Remove ultimo elemento;

Referências

- Baseado no material de aulas de:
  - ZIVIANI, Nivio. Projeto de algoritmos: com implementações em Java e C++. São Paulo: Thomson Learning, c2007. 621 p.
  - Para downloads dos códigos em Java:
  - http://www2.dcc.ufmg.br/livros/algoritmos-java/implementacoes-03.php

- Outras referências:
  - CORMEN, Thomas H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002. 916p. ISBN 8535209263 (Disponível no Acervo).

  - DROZDEK, Adam. Estrutura de dados e algoritmos em C++. São Paulo: Pioneira Thomson Learning, 2002. xviii, 579p. ISBN 8522102953 (Disponível no Acervo).